



Desafio Final — Prever Churn (Data Science)

Guia passo a passo com prompts de IA — instalar, configurar, construir e testar
Módulo 4 · Data Science e Testes · Projeto Social Garapuvu 2026 · Instrutor: Douglas Adriano Queiroz

Neste desafio você constrói um **pipeline completo** que prevê quais clientes vão **cancelar (churn)** — o problema mais clássico do mercado. Faça no VS Code com o **GitHub Copilot** (ou Claude): copie cada prompt na ordem, verifique o resultado e mostre o sistema rodando. Meta de qualidade: **dados validados, testes verdes e F1 ≥ 0,75**.

Duas trilhas: este guia é a trilha em **código (Python)**. Se preferir **no-code**, use o template `Desafio_Churn_Template.rmp` no RapidMiner (seção 8). O dado é o mesmo: `clientes.csv`.

1 Regras de ouro dos prompts

- **Dê contexto:** use `@workspace` e deixe abertos os arquivos relevantes.
- **Peça em partes:** uma função, um teste, um endpoint por vez.
- **Diga a stack e as regras:** "pandas + scikit-learn", "alvo = coluna cancelou", "F1 ≥ 0,75".
- **Peça os testes junto e rode-os** a cada passo.
- **Leia antes de aceitar** e faça `git commit` quando funcionar.

2 O que você vai construir

Etapa	O que faz	Como é testada
Carregar + validar	Lê o clientes.csv e valida schema/faixas	pandera / Great Expectations
Limpar / features	Padroniza e prepara colunas	Testes unitários (pytest)
Treinar modelo	scikit-learn prevê 'cancelou'	Métrica com limiar (F1 ≥ 0,75)
Servir por API	FastAPI expõe POST /prever	Teste de endpoint (pytest + requests)
Automatizar	CI roda tudo a cada push	GitHub Actions

Dados (clientes.csv): `id`, `tempo_casa_meses`, `plano`, `valor_mensal`, `uso_gb`, `chamados_suporte`, `cancelou` (alvo: `cancelou` = Sim/Não).

3 Instalar e verificar as ferramentas

```
# confirme que já tem:
$ python --version      # 3.10+ (ou python3 --version)
$ pip --version
$ code --version        # VS Code
# crie e ative um ambiente virtual (isola as libs do projeto):
$ python -m venv .venv
$ source .venv/bin/activate # Windows: .venv\Scripts\activate
```

Verificar: cada comando mostra uma versão; após ativar a venv, o terminal exibe `(.venv)` no início da linha. No VS Code, confirme a extensão **GitHub Copilot** logada e selecione o interpretador Python da `.venv` (Ctrl+Shift+P → "Python: Select Interpreter").

4 Bibliotecas do projeto

Biblioteca	Para quê
pandas	Ler e manipular a tabela de dados
scikit-learn	Treinar e avaliar o modelo
pandera	Validar o schema/faixas dos dados
fastapi + uvicorn	Servir o modelo por uma API
joblib	Salvar/carregar o modelo treinado
pytest + pytest-cov	Testes e cobertura
requests	Testar a API

```
$ pip install pandas scikit-learn pandera fastapi uvicorn joblib pytest pytest-cov requests
$ pip freeze > requirements.txt      # congela as versões
```

Verificar: `pip list` mostra as bibliotecas; o arquivo `requirements.txt` foi criado. Assim qualquer pessoa reinstala tudo com `pip install -r requirements.txt`.

5 Sequência de prompts (execute na ordem)

Fase 0 — Estrutura do projeto

PROMPT 0.1

@workspace Crie a estrutura de um projeto de Data Science para prever churn de clientes: pasta src/ (dados.py, limpeza.py, treino.py, api.py) e pasta tests/ (test_dados.py, test_limpeza.py, test_modelo.py, test_api.py), um requirements.txt e um README.md explicando o objetivo (prever a coluna 'cancelou' do clientes.csv). Ainda não implemente nada – só a estrutura e o README.

Verificar: as pastas e arquivos foram criados; o README descreve o desafio (dados, objetivo, meta $F1 \geq 0,75$).

Fase 1 — Carregar e validar os dados

PROMPT 1.1 — VALIDAÇÃO COM PANDERA

@workspace Em src/dados.py crie: (1) carregar(caminho) que lê o clientes.csv com pandas; (2) um schema pandera que valida: id inteiro e único; tempo_casa_meses inteiro ≥ 0 ; plano em {Basico, Padrao, Premium}; valor_mensal float > 0 ; uso_gb float ≥ 0 ; chamados_suporte inteiro ≥ 0 ; cancelou em {Sim, Nao}. (3) validar(df) que aplica o schema e lança erro se violar. Comente cada regra.

PROMPT 1.2 — TESTES DA VALIDAÇÃO

@workspace Em tests/test_dados.py gere testes com pytest: um DataFrame válido passa; um com plano inválido, valor_mensal negativo, ou cancelou fora de {Sim,Nao} deve falhar a validação (use pytest.raises).

Verificar: `pytest tests/test_dados.py -q` passa. A validação barra dado ruim antes de treinar.

Fase 2 — Limpeza e preparação (features)

PROMPT 2.1 — FUNÇÕES DE LIMPEZA + TESTES

@workspace Em src/limpeza.py crie funções puras e testáveis: padroniza_plano(df) (tira espaços e capitaliza a coluna plano); e prepara(df) que transforma 'cancelou' Sim/Nao em 1/0 e faz get_dummies na coluna plano. Em tests/test_limpeza.py, gere testes unitários com um DataFrame pequeno verificando a saída exata.

Verificar: `pytest tests/test_limpeza.py -q` passa; a saída bate célula a célula. Boa prática: teste as bordas (texto com espaço, maiúsculas).

Fase 3 — Treinar e avaliar o modelo

PROMPT 3.1 — TREINO COM MÉTRICA E LIMIAR

@workspace Em src/treino.py: carregue e valide (dados.py), prepare (limpeza.py), separe treino/teste com train_test_split (test_size=0.3, random_state=2001, stratify no alvo), treine um RandomForestClassifier(random_state=2001), calcule o F1 no teste e salve o modelo com joblib em modelo.joblib. Exponha uma função treinar_e_avaliar() que retorna o F1. Imprima o F1 e a matriz de confusão.

PROMPT 3.2 — TESTE DA MÉTRICA (THRESHOLD)

@workspace Em tests/test_modelo.py, gere um teste que chama `treinar_e_avaliar()` e faz `assert f1 >= 0.75`. Explique em comentário por que fixamos `random_state` (reprodutibilidade) e usamos `stratify` (evita divisão injusta das classes).

Verificar: `python -m src.treino` imprime o F1; `pytest tests/test_modelo.py -q` passa (F1 ≥ 0,75).
Cuidado com **data leakage**: não use o alvo dentro das features.

Fase 4 — Servir o modelo por API

PROMPT 4.1 — API FASTAPI

@workspace Em `src/api.py` crie uma API FastAPI que, no start, carrega `modelo.joblib`. Endpoint POST `/prever` recebe um JSON com `tempo_casa_meses`, `plano`, `valor_mensal`, `uso_gb`, `chamados_suporte`, aplica a mesma preparação (`limpeza.py`) e retorna `{'churn': 'Sim'/'Nao', 'probabilidade': float entre 0 e 1}`. Valide a entrada e retorne 400 se faltar campo. Comente o fluxo.

PROMPT 4.2 — TESTE DO ENDPOINT

@workspace Em tests/test_api.py, use o `TestClient` do FastAPI para testar: um POST `/prever` válido retorna 200 e uma probabilidade entre 0 e 1; um POST sem campos retorna 400.

```
# subir a API localmente:
$ uvicorn src.api:app --reload --port 8000
# abra http://localhost:8000/docs para testar pela interface do FastAPI
```

Verificar: a página `/docs` abre; um POST `/prever` devolve churn + probabilidade; `pytest tests/test_api.py -q` passa.

Fase 5 — Rodar tudo, cobertura e CI

```
$ pytest --cov=src --cov-report=term-missing -q # todos os testes + cobertura
```

PROMPT 5.1 — CI NO GITHUB ACTIONS

@workspace Crie `.github/workflows/ci.yml` que, a cada push: instala o `requirements.txt`, roda `pytest` com cobertura e falha se algum teste falhar. Use Python 3.11.

Verificar: a suíte inteira fica verde (validação de dado, limpeza, modelo e API) e o relatório de cobertura aparece. Boa prática: cobertura mostra o que faltou testar — não prova ausência de bug.

6 Alternativa no-code: Altair AI Studio (antigo RapidMiner)

Não quer codar? Faça o mesmo desafio de forma visual. O **Altair AI Studio** é o antigo **RapidMiner** (a Altair comprou o RapidMiner em 2022; a Siemens comprou a Altair em 2025) — o motor é o mesmo.

Instalar no Mac: abra o `.dmg` baixado e **arraste o ícone "AI Studio" para a pasta Applications**; abra pelo Launchpad (se bloquear: Ajustes → Privacidade e Segurança → "Abrir mesmo assim").

- No AI Studio: **File** → **Import Process** → escolha `Desafio_Churn_Template.rmp` (ou arraste o `.rmp` para a área de processo).
- No operador **"1 - Ler clientes.csv"**, em *Parameters* → *csv_file*, aponte para o `clientes.csv`.
- Clique em **Run** (▶) e leia o **F1** no operador **"7 - Desempenho"** (meta $\geq 0,75$).

Passo a passo detalhado (com print da instalação): veja `README_Altair_AI_Studio.md` na pasta `Desafio Churn - RapidMiner`.

Para apresentar aos alunos: mostre o pipeline rodando — a validação barrando um dado inválido, os testes ficando verdes, a API devolvendo a previsão de churn e a métrica acima do limiar. É o mesmo raciocínio do Desafio de QA, agora com dados.

7 Checklist de entrega

Item	Ok?
Repositório no GitHub (código ou processo RapidMiner) + README	☐
Dados validados (pandera/GE) barrando dado inválido	☐
Testes verdes: dados, limpeza, modelo e API (no CI)	☐
$F1 \geq 0,75$ no conjunto de teste (com seed fixa)	☐
API POST /prever respondendo churn + probabilidade	☐
Apresentação: pipeline rodando e decisões explicadas	☐

8 Referências

pandas / scikit-learn	pandas.pydata.org · scikit-learn.org
pandera / Great Expectations	union.ai/pandera · greatexpectations.io
FastAPI / pytest	fastapi.tiangolo.com · docs.pytest.org
RapidMiner / Dataset	rapidminer.com · "Telco Customer Churn" (Kaggle)